СЕМИНАР ПО ЛЕКЦИИ 5

dbt: аналитический слой магазина в Greenplum

В начале — краткая теория по лекции. Затем задания: запускаем стенд, пишем модели staging, покрываем их тестами, собираем в хранилище Greenplum первую витрину заказов и документацию. Весь код — в репозитории `github.com/RKurbanova/store_hse`, подробная инструкция по установке — в методичке «Как запустить проект семинара».

КРАТКАЯ ТЕОРИЯ ПО ЛЕКЦИИ

1 ELT и место dbt

Раньше данные сначала обрабатывали, потом загружали в хранилище (**ETL**). Сейчас хранилища мощные, поэтому сырые данные загружают как есть, а преобразуют уже внутри (**ELT**). dbt отвечает только за букву **T**: он не забирает данные из источников, а превращает уже загруженные таблицы в витрины.

- **Модель** — файл с одним `select`. dbt сам оборачивает его в `create view` или `create table`.
- `{{ source('shop', 'orders') }}` — ссылка на сырую таблицу источника.
- `{{ ref('stg_shop__orders') }}` — ссылка на другую модель. По этим ссылкам dbt строит граф и сам определяет порядок сборки.

Где что лежит у нас

```
PostgreSQL (OLTP, порт 5436)            Greenplum (хранилище, порт 5440, база dwh)
база shop, схема public ── loader ──►  схема raw ── dbt ──► analytics_staging
                           (E + L)                  (T)     analytics_intermediate
                                                            analytics_marts
```

loader копирует сырые таблицы магазина в Greenplum, в схему `raw` — это загрузка, буквы E и L. Позже её будет запускать Airflow. **dbt** работает только внутри Greenplum: читает `raw` и строит слои `analytics_*`. Данные между базами dbt не переносит.

2 Три слоя проекта

Слой	Префикс	Что делает	Чего там нет
Staging	`stg_`	Одна модель на таблицу источника: переименовать колонки, привести типы, простые флаги	JOIN-ов и агрегаций
Intermediate	`int_`	Промежуточные шаги: JOIN-ы, группировки, сложная логика	Обращений к источникам
Marts	`fct_`, `dim_`	Готовые витрины для бизнеса: факты и измерения	Технического мусора

Зернистость — что означает одна строка модели: «один заказ», «один товар в заказе». Её определяют до того, как писать код. Главная ошибка — соединить заказы с платежами до группировки: заказ с

двумя платежами раздвоится, и выручка вырастет.

3 Материализации

Вид	Что dbt создаёт в базе	Когда использовать
`view`	Представление: запрос выполняется при каждом обращении	Staging, лёгкие промежуточные шаги
`table`	Таблица, которая пересоздаётся при каждом запуске	Витрины, которые часто читают
`incremental`	Таблица, в которую дописываются только новые строки	Большие факты: события, логи
`ephemeral`	Ничего: модель подставляется в другие как подзапрос	Мелкие вспомогательные шаги

Материализацию задают для целой папки в `dbt_project.yml` (`+materialized: view`) или в самой модели: `{{ config(materialized='table') }}`.

Таблица в Greenplum ещё и распределяется по сегментам, как в практике 3. В нашем проекте это задаётся там же, в `config()`: `distributed_by='order_id'`, `distributed_replicated=true` для маленьких справочников, `orientation='column'` для колоночного хранения. Эти настройки понимает готовый макрос `macros/greenplum_create_table_as.sql`.

4 Тесты

Тест в dbt — это запрос, который ищет «плохие» строки. **Вернул 0 строк — тест пройден**, вернул хоть одну — упал. Проверяются не программы, а данные.

- **Generic-тесты** — готовые шаблоны, подключаются строкой в YAML: `unique`, `not_null`, `accepted_values` (только разрешённые значения), `relationships` (ключ есть в другой таблице).
- **Singular-тест** — свой SQL-файл в папке `tests/`, который выбирает нарушителей бизнес-правила.
- `dbt build` собирает модели и сразу их тестирует. Если тест упал, зависимые модели получают статус `SKIP`: плохие данные дальше не идут.

5 Документация, Jinja и макросы

- Описания моделей и колонок пишутся в YAML рядом с тестами (`description`). `dbt docs generate` собирает из них сайт с графом зависимостей (lineage).
- **Jinja** — шаблоны внутри SQL: `{{ ... }}` подставляет значение, `{% ... %}` — логика (циклы, условия), `{# ... #}` — комментарий. dbt сначала генерирует из шаблона обычный SQL, потом отправляет его в базу. Готовый SQL лежит в папке `target/compiled`.
- **Макрос** — функция на Jinja, которая возвращает кусок SQL. Её можно вызывать в разных моделях.

6 Главные команды

Команда	Что делает
`dbt run -s имя_модели`	Собрать одну модель
`dbt run -s staging`	Собрать все модели папки
`dbt run -s +fct_orders`	Собрать модель и всё, от чего она зависит
`dbt test -s имя_модели`	Прогнать тесты одной модели
`dbt build`	Собрать и протестировать всё в правильном порядке
`dbt compile -s имя_модели`	Только сгенерировать SQL в `target/compiled`

У нас dbt работает в Docker, поэтому полная форма любой команды — `docker compose run --rm dbt …`. Сокращение для macOS и Linux: `alias dbt="docker compose run --rm dbt"`, для PowerShell: `function dbt { docker compose run --rm dbt @args }`.

ЗАДАНИЯ

0 Подготовка стенда

Нужны Git, запущенный Docker Desktop и DBeaver. Стенд практики 3 остановите: два Greenplum не поместятся в память. Если что-то не устанавливается, смотрите методичку «Как запустить проект семинара»: там шаги для macOS, Windows и Linux и таблица частых ошибок.

```
git clone https://github.com/RKurbanova/store_hse.git
cd store_hse
docker compose up -d              # PostgreSQL, Greenplum и загрузка raw
docker compose logs loader        # ждём строку «Готово» и число строк
docker compose build dbt          # первый раз 1–3 минуты
docker compose run --rm dbt debug # в конце: All checks passed!
```

Подключение DBeaver	Порт	База / пользователь / пароль	Что там
OLTP (PostgreSQL)	`5436`	`shop` / `student` / `studentpass`	Рабочая база магазина, схема `public`
DWH (Greenplum)	`5440`	`dwh` / `gpadmin` / пустой	Сырой слой `raw` и всё, что строит dbt

1 Первый запуск

В проекте уже есть описание источников и одна готовая модель — `stg_shop__orders`. Остальные модели — заготовки с пропусками `TODO`. Пока пропуски не заполнены, модель падает с ошибкой `column "todo" does not exist`: так и задумано.

1.1 Сравните источник и хранилище

В DBeaver откройте `public.orders` в OLTP и `raw.orders` в DWH. В обеих по 141 строке: loader скопировал таблицу как есть.

1.2 Соберите готовую модель

```
dbt run -s stg_shop__orders
```

1.3 Найдите её в хранилище

В DBeaver обновите подключение DWH и откройте схему `analytics_staging`. Там появилось представление `stg_shop__orders` на 141 строку.

1.4 Посмотрите, что dbt отправил в базу

Откройте файл `target/compiled/dbt_shop/models/staging/shop/stg_shop__orders.sql` и сравните его с исходным файлом модели.

Вопросы для обсуждения:

- Во что превратилась строка `{{ source('shop', 'orders') }}`?
- Почему модель стала представлением, а не таблицей? Где это задано?
- Почему схема называется `analytics_staging`? Подсказка: файлы `profiles.yml` и `dbt_project.yml`.

2 Модели staging

Заполните `TODO` в трёх файлах папки `models/staging/shop/`. Помните правило слоя: одна таблица источника — одна модель, без JOIN и агрегаций.

Модель	Что дописать
`stg_shop__customers`	`registration_date` — дата регистрации без времени
`stg_shop__order_items`	`order_item_id` — ключ вида `101-1` из `order_id` и `product_id`; `gross_amount` — сумма до скидки; `line_amount` — сумма после скидки
`stg_shop__payments`	`is_refund` — `true` для возврата. Возврат записан в источнике успешной операцией с отрицательной суммой

```
dbt run -s staging     # в схеме analytics_staging должно стать 4 представления
```

3 Тесты

3.1 Допишите generic-тесты

В файле `models/staging/shop/_shop__models.yml` для `stg_shop__orders` часть тестов уже есть. Допишите остальные по образцу:

- `unique` и `not_null` на ключ каждой модели: `customer_id`, `order_item_id`, `payment_id`;
- `relationships`: `customer_id` заказа есть в `stg_shop__customers`, `order_id` позиций и платежей есть в `stg_shop__orders`;
- `accepted_values` для `payment_status`: `success`, `failed`, `pending`;
- `accepted_values` для `order_status`. По документации OLTP-команды статусов четыре: `paid`, `pending`, `returned`, `cancelled`.

```
      - name: customer_id
        data_tests:
          - not_null
          - relationships:
              arguments:
                to: ref('stg_shop__customers')
                field: customer_id
      - name: payment_status
        data_tests:
          - accepted_values:
              arguments:
                values: ['success', 'failed', 'pending']
```

3.2 Запустите тесты

```
dbt test -s staging
```

Два теста упадут — это часть задания. «Плохие» строки dbt сохраняет в схему `analytics_dbt_test__audit`, а готовый запрос для их просмотра печатает после строки `See test failures:`. Выполните этот запрос в DBeaver.

Вопросы для обсуждения:

- Какое значение статуса нашёл тест `accepted_values`? Ошибка в данных или в нашем представлении о данных?
- Что найдёт тест из лекции `assert_no_negative_payments`? Верно ли само правило «платёж не может быть отрицательным»?
- Что в каждом случае исправить — данные, модель или тест?

3.3 Напишите правильный singular-тест

Удалите `tests/assert_no_negative_payments.sql` и создайте `tests/assert_refunds_only_for_returned_orders.sql`. Правило: возврат допустим только у заказа со статусом `returned`.

```
select p.payment_id, p.order_id, p.amount, o.order_status
from {{ ref('stg_shop__payments') }} as p
join {{ ref('stg_shop__orders') }} as o
    on o.order_id = p.order_id
where /* TODO */
```

4 Первая витрина заказов

Правило из практики 2: **подтверждённая покупка** — заказ со статусом `paid` и хотя бы одной успешной оплатой. Возврат оплатой не считается.

4.1 Заполните intermediate-модель

Файл `models/intermediate/int_orders_with_payments.sql`: одна строка — один заказ, к нему пристыкованы итоги по платежам. Сначала сверните платежи до заказа в CTE `order_payments`, потом присоедините их к заказам.

4.2 Исключите тестовые заказы из витрины

Файл `models/marts/fct_orders.sql`: допишите условие `where`.

4.3 Опишите витрину и соберите проект

Создайте `models/marts/_marts__models.yml` с описанием `fct_orders`. Добавьте тесты `unique` и `not_null` на `order_id` и `not_null` на `is_confirmed`.

```
dbt build
```

4.4 Распределите витрину по сегментам Greenplum

В подключении DWH посмотрите, как сейчас хранится витрина:

```
select pg_get_table_distributedby('analytics_marts.fct_orders'::regclass);
select gp_segment_id, count(*) from analytics_marts.fct_orders group by 1 order by 1;
```

Без настроек она `DISTRIBUTED RANDOMLY`. Выберите ключ распределения и добавьте первой строкой в `fct_orders.sql`:

```
{{ config(distributed_by='TODO') }}
```

Выполните `dbt run -s fct_orders` и оба запроса ещё раз.

Вопросы для обсуждения:

- Сколько строк получится у заказа с двумя платежами, если соединить заказы с платежами до группировки?
- Что случится с заказом 119, если вместо `left join` написать `join`?
- Почему staging и intermediate — представления, а витрина — таблица?
- По какому столбцу вы распределили витрину и почему? Подсказка: как распределены `raw.order_items` и `raw.payments`? Загляните в `init/greenplum/load_raw.sql`.
- Что будет, если распределить витрину по `order_status`?

5 Документация

Допишите `description` для моделей и ключевых колонок, затем:

```
dbt docs generate
docker compose run --rm --service-ports dbt docs serve --host 0.0.0.0 --port 8080 --no-browser
```

Откройте `http://localhost:8080`. Кнопка графа зависимостей — в правом нижнем углу. Найдите путь от `source.shop.payments` до `fct_orders`. Остановить сервер — `Ctrl+C`.

Бонус: цикл Jinja

В CTE `order_payments` добавьте число операций в каждом статусе: `success_operations_count`, `failed_operations_count`, `pending_operations_count`. Напишите не три строки вручную, а цикл `{% for status in [...] %}`. Результат проверьте командой `dbt compile`.

6 Самопроверка

Проверка	Ожидаемый результат
Представления в `analytics_staging`	4
`accepted_values` с четырьмя статусами	Падает: 1 лишний статус
`assert_no_negative_payments`	Падает: 7 строк — это возвраты
Строки в `analytics_marts.fct_orders`	`135` — из 141 заказа исключены 6 тестовых
Подтверждённые заказы / их сумма	`106` / `91 620,00` — как в практике 2
Заказ `113`	`order_amount = 200`, но `paid_amount = 400`: два успешных платежа
Заказ `107`	`returned`, оплачено `700`, возвращено `700`, `net_paid_amount = 0`
Заказ `119`	Статус `paid`, но `is_confirmed = false`: оплаты нет
Распределение `fct_orders`	`DISTRIBUTED BY (order_id)`, строки на всех 4 сегментах примерно поровну
Итог `dbt build`	`ERROR=0`

Подтверждённых заказов 107? Проверьте, требуете ли вы успешную оплату. Строк больше 135 или сумма больше 91 620,00? Платежи нужно свернуть до заказа до JOIN.

7 Результат семинара

Сохраните работу в Git — от неё начинается домашнее задание:

```
git add .
git commit -m "Семинар 5: staging, int_orders_with_payments, fct_orders в Greenplum"
```

В домашнем задании вы достроите аналитический слой в Greenplum по схеме «звезда»:

- факты `fct_orders` и `fct_order_items`, измерения `dim_customers`, `dim_products` с иерархией категорий и календарь `dim_dates`;
- для каждой витрины — способ хранения в Greenplum (распределение, колоночное хранение) с обоснованием;
- тесты на ключи и связи, свой generic-тест и минимум 4 singular-теста на бизнес-логику;
- документацию: описания, doc-блок, ER-диаграмму звезды и скриншот графа зависимостей;
- сдача — ссылка на ваш репозиторий; `dbt build` должен проходить без ошибок.